RegRadar — Docker Setup Guide

Phase 1 MVP · Step-by-Step

Before You Start — Prerequisites

Install these on your machine first. Check each one:

```bash
docker --version          # Need 24.0+
docker compose version    # Need 2.20+
python --version          # Need 3.12+ (for local scripts only)
git --version             # Any version
```

Don't have Docker? Download Docker Desktop from → https://docs.docker.com/get-docker/ It installs both `docker` and `docker compose` together.

Step 1 — Project Folder Structure

Create this folder structure on your machine. Everything lives here.

```
regradar/
├── backend/
│   ├── app/
│   │   ├── __init__.py
│   │   └── main.py           ← we create this in Step 4
│   ├── Dockerfile            ← copy from the files provided
│   └── requirements.txt      ← copy from the files provided
├── docker-compose.yml        ← copy from the files provided
├── .env.example              ← copy from the files provided
└── .env                      ← you create this in Step 3
```

Run these commands to create the folders:

```bash
mkdir -p regradar/backend/app
cd regradar
touch backend/app/__init__.py
```

Then copy the provided files into place:

- `docker-compose.yml` → `regradar/docker-compose.yml`
- `Dockerfile` → `regradar/backend/Dockerfile`
- `requirements.txt` → `regradar/backend/requirements.txt`
- `.env.example` → `regradar/.env.example`

Step 2 — Add RegRadar to .gitignore

Before anything else, protect your secrets:

bash

```
# Inside regradar/
cat > .gitignore << 'EOF'
# Environment secrets — NEVER commit this
.env

# Python
__pycache__/
*.pyc
*.pyo
.venv/

# Docker volumes (local data)
postgres_data/
redis_data/

# OS
.DS_Store
Thumbs.db
EOF
```

Step 3 — Set Up Your .env File

bash

```
# Copy the example file
cp .env.example .env
```

Now open `.env` and fill in two things:

1. Your Anthropic API Key Get it from https://console.anthropic.com → API Keys → Create Key

```
ANTHROPIC_API_KEY=sk-ant-api03-xxxxxxxxxxxxxxx
```

2. Generate Secret Keys (run this in your terminal):

```bash
python -c "import secrets; print('SECRET_KEY=' + secrets.token_hex(32))"
python -c "import secrets; print('WEBHOOK_SIGNING_SECRET=' + secrets.token_hex(32))"
```

Copy the output values into your `.env` file.

> ✅ Leave `DATABASE_URL` and `REDIS_URL` as-is — Docker handles those automatically.

Step 4 — Create the Minimal FastAPI App

You need a bare-minimum `main.py` so the API container can start. Create `backend/app/main.py`:

```python
from fastapi import FastAPI

app = FastAPI(
    title="RegRadar API",
    description="Regulatory & Compliance Change Monitoring",
    version="0.1.0"
)

@app.get("/health")
def health():
    return {"status": "ok", "service": "regradar-api"}
```

This is enough to get the container running. You'll add real endpoints on top of this.

Step 5 — Build and Start Everything

```bash
```

```
# Make sure you're in the regradar/ folder (where docker-compose.yml is)
cd regradar

# Build all Docker images (takes 3-5 minutes the first time — Playwright is large)
docker compose build

# Start all services in the background
docker compose up -d
```

You'll see Docker pull images and build your app. When it finishes, all 5 services are running.

Step 6 — Verify Everything Is Running

bash

```
docker compose ps
```

You should see all services with status `running` or `healthy`:

```
NAME                STATUS
regradar_api        running (healthy)
regradar_worker     running
regradar_beat       running
regradar_db         running (healthy)
regradar_redis      running (healthy)
```

Test the API is alive:

bash

```
curl http://localhost:8000/health
# Expected: {"status":"ok","service":"regradar-api"}
```

Or open in browser: http://localhost:8000/docs You'll see the auto-generated Swagger UI — this is your interactive API docs.

Step 7 — Run Database Migrations

The database is running but has no tables yet. Run Alembic to create them.

First, set up Alembic (one time only):

```bash
docker compose exec api alembic init alembic
```

This creates an `alembic/` folder inside your backend. You'll configure it when you add your database models (SQLAlchemy).

Once you have models, run migrations like this:

```bash
# Generate a migration from your models
docker compose exec api alembic revision --autogenerate -m "initial tables"

# Apply the migration to the database
docker compose exec api alembic upgrade head
```

> 💡 You don't need to do this on Day 1 — come back here when you've written your SQLAlchemy models.

Step 8 — Test the Database Connection

Verify Postgres is accessible:

```bash
# Open a Postgres shell inside the container
docker compose exec db psql -U regradar -d regradar

# Inside the psql shell, type:
\l        # list databases
\q        # quit
```

Or connect with a GUI like TablePlus, DBeaver, or pgAdmin:

- Host: `localhost`
- Port: `5432`
- User: `regradar`
- Password: `regradar_pass`
- Database: `regradar`

Step 9 — Test Redis

bash

```
# Open a Redis shell
docker compose exec redis redis-cli

# Inside redis-cli, type:
ping        # Should return: PONG
quit        # Exit
```

Daily Workflow — Commands You'll Use Every Day

bash

```bash
# Start everything (after first setup)
docker compose up -d

# Stop everything (data is preserved)
docker compose down

# View logs for all services
docker compose logs -f

# View logs for just the API
docker compose logs -f api

# View logs for the Celery worker
docker compose logs -f worker

# Restart just the API (after code changes that don't hot-reload)
docker compose restart api

# Rebuild after changing requirements.txt or Dockerfile
docker compose build api
docker compose up -d api

# Open a shell inside the API container
docker compose exec api bash

# Run a one-off Python command inside the container
docker compose exec api python -c "print('hello from inside Docker')"
```

Troubleshooting

Problem: `api` container keeps restarting

bash

```bash
# Check what's going wrong
docker compose logs api
```

Most common causes:

- `main.py` has a syntax error — fix it, then `docker compose restart api`

- `.env` file is missing or has a wrong value — check it exists and `ANTHROPIC_API_KEY` is filled in
- Port 8000 is already used by something else — change `"8000:8000"` to `"8001:8000"` in docker-compose.yml

---

Problem: Database connection error

```bash
# Check if db is healthy
docker compose ps db

# If not healthy, check logs
docker compose logs db
```

Most common cause: the `api` container started before `db` was ready. The `depends_on` with `service_healthy` should prevent this, but if it happens:

```bash
docker compose restart api worker beat
```

Problem: `docker compose build` is very slow

Normal on first build — Playwright downloads ~300MB of Chromium.
Subsequent builds use the Docker cache and take 10–30 seconds.

If you want to skip Playwright for now (you won't scrape JS-rendered sites yet):

```bash
# Comment out the Playwright install line in Dockerfile temporarily:
# RUN playwright install chromium --with-deps
```

Problem: Port already in use

```bash
```

```
# Find what's using port 8000
lsof -i :8000      # macOS/Linux
netstat -ano | findstr :8000   # Windows

# Or just change the port in docker-compose.yml
ports:
  - "8001:8000"   # API now at http://localhost:8001
```

Nuke everything and start fresh

Only do this if something is badly broken. **Deletes all database data.**

bash

```
docker compose down -v          # Stops containers AND deletes volumes
docker compose build --no-cache # Rebuilds images from scratch
docker compose up -d
```

What Each Service Does — Quick Reference

Service	Container Name	What It Does	Port
`api`	regradar_api	FastAPI — serves your REST endpoints	8000
`worker`	regradar_worker	Celery — runs scraping + webhook jobs	—
`beat`	regradar_beat	Celery Beat — triggers scraper every 15 min	—
`db`	regradar_db	PostgreSQL — stores all data	5432
`redis`	regradar_redis	Redis — task queue + cache	6379

Next Steps After Setup

Once all services are green:

1. Write your SQLAlchemy models (`api_keys`, `subscriptions`, `changes`, `webhook_deliveries`)

2. Run `alembic revision --autogenerate` to create migration files
3. Run `alembic upgrade head` to create the tables
4. Build `POST /subscriptions` endpoint in FastAPI
5. Build `GET /subscriptions` endpoint
6. Tell your frontend teammate — base URL is `http://localhost:8000/v1`
